

REPONDRE SUR LA FEUILLE DE REPONSE

Exercice 1 : (5 points)

Partie 1: Relier chaque concept de la POO avec sa définition.

Concepts	Définition
1. Classe interne locale	a. Une autre façon de passer un nombre variable de paramètres de même type à une méthode.
2. Exception contrôlé	b. Est la classe de base de tous les layouts panes. Placer comme panneau, il n'applique aucune disposition spéciale à ses nœuds enfants.
3. Radio Button	c. Elle peut être implémentée en utilisant une classe interne anonyme.
4. Pane	d. Elle correspond à une implémentation automatique d'une classe anonyme et à un renvoi sur une méthode de votre choix.
5. Classe interne non statique	e. Générer quand une action de la souris s'est produite dans un composant.
6. Référence sur méthode	f. C'est une façon concise de créer une instance d'une interface fonctionnelle en implémentant sa méthode.
7. Interface fonctionnelle	g. Représente un bouton bistable. Il comporte donc deux états : un clic le met à l'état sélectionné (on), un nouveau clic le remet à l'état désélectionné (off).
8. Varargs	h. Elle peuvent être standard ou bien personnalisée et doivent être traitées ou propagées.
9. MouseEvent	i. Elle est définie à l'intérieur d'une autre classe en tant que membre de cette dernière.
10. Expression lambda	j. Elle est définie à l'intérieur d'une méthode.

Partie 2: Pour chacune des propositions, répondre par vrai (V) ou faux (F).

N°	Proposition
1.	L'interface de marquage n'impose pas la définition d'une méthode, mais indique que les objets qui les implémentent ont certaines propriétés.
2.	La définition du type énuméré peut inclure un constructeur de visibilité publique.
3.	Il n'est pas possible d'ajouter des champs, des constructeurs et des méthodes dans une classe de type énuméré.
4.	C'est possible d'instancier un objet d'un type paramétré.
5.	Étant donné que o1 est un référent déclaré et initialisé vers un objet de la classe C<T>, o2 est un référent déclaré et initialisé vers un objet de la classe C<U>, o1= o2 est correcte implique toujours que U est un sous type de T.
6.	La méthode statique values() retourne un tableau des constantes d'énumération dans leur ordre de déclaration.
7.	On ne peut pas instancier de tableaux d'éléments d'un type générique.
8.	Colors.valueOf("White") lève l'exception IllegalArgumentException si la chaîne White ne peut pas être convertie en type Colors.
9.	La méthode setPadding() est utilisée pour définir la même marge entre le bord du conteneur et les composants enfants.
10.	FlowPane insère tous les nœuds enfants en une seule rangée horizontale.

Exercice 2: (10 points)

Partie 1:

- Créer une énumération appelée Continent représentant les continents du monde (AFRIQUE, AMERIQUE, ASIE, EUROPE, OCEANIE).
- Ajouter un attribut à chaque valeur énumérée pour représenter la superficie (double) du continent.
- Rajouter le constructeur pour initialiser l'attribut superficie, getSuperficie() et setSuperficie().
- Dans un programme principal, afficher les informations de chaque continent ainsi que la superficie totale de tous les continents.

Partie 2: Soit la classe Pays définit en utilisant le type énuméré Continent comme suit:

<pre> class Pays implements Comparable<Pays>{ private final Continent continent; private String nom, capitale; private double superficie; private int population; public Pays(Continent continent, String nom, String capitale, double superficie, int population) { this.continent = continent; this.nom = nom; this.capitale = capitale; this.superficie = superficie; this.population = population; } public Continent getContinent() {return continent;} public String getNom() {return nom;} public void setNom(String nom) {this.nom = nom;} public String getCapitale() {return capitale;} public void setCapitale(String capitale) { this.capitale = capitale; } </pre>	<pre> public double getSuperficie() { return superficie;} public void setSuperficie(double superficie) { this.superficie = superficie;} public int getPopulation() {return population;} public void setPopulation(int population) { this.population = population; } public String toString() { return "Pays{" + "nom=" + nom + "\" + ", capitale=" + capitale + "\" + ", population=" + population + ", superficie=" + superficie + ", continent=" + continent + '}''; } public int compareTo(Pays p) { return nom.compareToIgnoreCase(p.nom);} } </pre>
<pre> public class AppGeog { public static void main(String[] args) { List<Pays> listePays =new LinkedList<>(); //Ajouter des pays à la liste listePays.add(new Pays(Continent.AFRIQUE,"Algérie", "Alger", 43000000, 2381741)); listePays.add(new Pays(Continent.ASIE,"Palestine", "Ramallah", 5000000, 6020)); }} </pre>	

Question: En créant une List<Pays> **listePays**, comme illustrer dans la classe **AppGeog**, pour chaque traitement du tableau ci-dessous, associer le numéro de la portion (ou les portions) du code (du tableau des codes) qui réalise correctement la tâche décrite.

Traitement	
a. Afficher les pays de la liste triée selon l'ordre naturel.	(4 et 15, 14)
b. Calculer la population totale de tous les pays de la liste.	(5,12, 16)
c. Afficher les noms de pays dans l'ordre inverse de la liste.	(11)
d. Remplacer les noms de toutes les capitales par leur version en majuscules.	(3)
e. Donner le pays de la liste qui la plus grande population.	(7)
f. Afficher les noms de pays dont la population est supérieure à un certain seuil donné	(10,6)
g. Supprimer définitivement de la liste les pays dont la superficie est inférieure à un certain seuil donné	(2)
h. Trouver tous les pays d'un continent spécifique.	(17,9)
i. Afficher les pays dont le nom de la capitale contient la chaîne str.	(19)
j. Compter le nombre de pays d'un continent donné dont superficie est supérieur à un certain seuil donné.	(13)

N°	Le code	N°	Le code
1	listePays.forEach(pays -> System.out.println(pays));	2	ListIterator<Pays> iterator = listePays.listIterator(); while (iterator.hasNext()) { Pays pays = iterator.next(); if (pays.getSuperficie() < s) iterator.remove();}
3	ListIterator <Pays> iterator = listePays.listIterator(); while (iterator.hasNext()) { Pays pays=iterator.next(); pays.setCapitale(pays.getCapitale().toUpperCase());}	4	Collections.sort(listePays); for (Pays pays : listePays) { out.println(pays); }
5	out.println(listePays.stream().mapToInt(p -> p.getPopulation()).sum());	6	listePays.stream().filter(p -> p.getPopulation()>s).forEach(p -> out.println(p.getNom()));
7	out.println(listePays.stream() .max((p1,p2) -> Integer.compare(p1.getPopulation(), p2.getPopulation())).get());	8	Iterator<Pays> iterator = listePays.iterator(); while (iterator.hasNext()) out.println(iterator.next().getNom());

9	for(Pays p: listePays){ if(p.getContinent().equals(Continent.valueOf(str.toUpperCase()))){ out.println(p);} }	10	ListIterator <Pays> iterator = listePays.listIterator(); while (iterator.hasNext()) { Pays pays = iterator.next(); if (pays.getPopulation() > s) out.println(pays.getNom()); } }
11	ListIterator<Pays> iterator = listePays.listIterator(listePays.size()); while (iterator.hasPrevious()) out.println(iterator.previous().getNom());	12	int s=0; for (Pays pays : listePays) s += pays.getPopulation(); out.println(s);
13	out.println(listePays.stream().filter(p -> p.getContinent().equals(Continent.valueOf("asie".toU pperCase()))).filter(p-> p.getSuperficie() > 10000000).count());	14	Collections.sort(listePays, Pays::compareTo); listePays.forEach(pays -> out.println(pays));
15	listePays.stream() .sorted().forEach(out::println);	16	ListIterator<Pays> iterator = listePays.listIterator(); double s=0; while (iterator.hasNext()) s+=iterator.next().getSuperficie (); out.println(s);
17	listePays.stream().filter(p -> p.getContinent().equals(Continent.AFRIQUE)) .forEach(out::println);	18	Iterator<Pays> iterator = listePays.iterator(); while (iterator.hasNext()) { Pays pays=iterator.next(); pays.setCapitale(pays.getCapitale() .toLowerCase ());}
19	listePays.stream().peek(p -> p.setCapitale(p.getCapitale().toLowerCase())).filter(p -> p.getCapitale().contains("ma")).forEach(out::println);	20	listePays.stream().filter(p->p.getSuperficie()>s);

Exercice 3: (5 Points) Analyseur de texte

Soit le code ci-dessous d'une application en JavaFx qui réalise un index pour compter le nombre d'occurrence des mots dans un texte. L'index est implémenté en utilisant un Map<String, Integer>. La clé (String) du Map est un mot du texte. Alors que la valeur (Integer) représente le nombre d'occurrence de chaque mot stocké comme clé.

```
public class GuiTexAnaly extends Application{
    public static List<String> texteToList(String str){
        String[] tabmots = str.split("\\s+"); List<String> Listmot =new ArrayList<>();
        for (String mot : tabmots) {
            mot = mot.toLowerCase().replaceAll("[^a-zA-Z0-9]", "");
            if (!mot.isEmpty()) Listmot.add(mot);
        }
        return Listmot;
    }
    public static Map<String,Integer> getIndex(String str){
        List<String> Listmots=texteToList(str); Map<String, Integer> indexMap = new HashMap<>();
        for (String mot : Listmots) {
            if(indexMap.containsKey(mot)) indexMap.replace(mot,indexMap.get(mot)+1 );
            else indexMap.put(mot, 1);
        }
        return indexMap;
    }
    public static String getMotMaxFrequence(Map<String, Integer> Index) {
        String maxMot = null; int max = 0;
        for (Map.Entry<String, Integer> entry : Index.entrySet()) {
            String mot = entry.getKey(); int cpt = entry.getValue();
            if (cpt > max) { max = cpt; maxMot = mot; }
        }
        return maxMot;
    }
}
```

```

public static void afficherMap(Map<String,Integer> map){
    for (Map.Entry<String, Integer> entry : map.entrySet())
        out.println(entry.getKey() + ": " + entry.getValue());
}
public void start(Stage stg) {
    stg.setTitle("Analyseur de texte");
    VBox root=new VBox();
    root.setSpacing(10);
    root.setPadding(new Insets(10));
    Label LtexIn=new Label("Texte à analyser");
    TextArea tex=new TextArea();
    Button btnAnalyser =new Button("Analyser");
    HBox root1 = new HBox();
    root1.setPadding(new Insets(10));
    root1.setSpacing(10);
    Label LTitrenbMot= new Label("Nombre de mots: ");
    Label LnbMot= new Label("xxx ");
    root1.getChildren().addAll(LTitrenbMot,LnbMot);
    HBox root2 = new HBox();
    root2.setPadding(new Insets(10));
    root2.setSpacing(10);
    Label LtitreMotmax= new Label("Le mot le plus fréquent: ");
    Label LnbMotmax= new Label("xxx ");
    root2.getChildren().addAll(LtitreMotmax,LnbMotmax);
    root.getChildren().addAll(LtexIn,tex,btnAnalyser,root1,root2);
    btnAnalyser.setOnAction( /*Compléter */ );
    Scene scene = new Scene(root); stg.setScene(scene); stg.show();
}
public static void main(String[] args) { launch(args); }}

```

Le tableau suivant indique le rôle des méthodes de manipulation de l'index sous forme de Map<String,Integer>:

Méthode	Rôle
static List<String> texteToList(String str)	Génère une liste de mot à partir d'une chaîne de caractères après avoir effectué un nettoyage des mots du texte.
static Map<String,Integer> getIndex(String str)	En utilisant texteToList() , getIndex() génère un index sous forme de Map<String,Integer>
static String getMotMaxFrequence(Map<String,Integer> Index)	Retourne le mot le plus fréquent de l'index.
static void afficherMap(Map<String,Integer> map)	Affiche le contenu de l'index (Map<String, Integer> : mot : nombre d'occurrence)

1. Dessiner l'interface graphique de l'application. **2pts**

2. Implémenter le gestionnaire l'événement du bouton **btnAnalyser** afin de faire les traitements suivants: **3pts**

- Afficher le contenu de l'index du texte sur la console.
- Afficher le nombre des mots de l'index dans le label : **LnbMot**.
- Afficher le mot le plus fréquent dans le label : **LnbMotmax**.

Bonus : Rajouter à l'interface un composant de type Combox (Liste déroulante) pour afficher les mots de l'index (les clés du Map).

Label	String getText(), void setText(String text)
TextArea	String getText(), void setText(String text)
ComboBox	ComboBox(), void getSelectionModel().select(int i); combobox.getItems().add(String str); combobox.getItems().addAll(Collection L); String getSelectionModel().getSelectedItem();